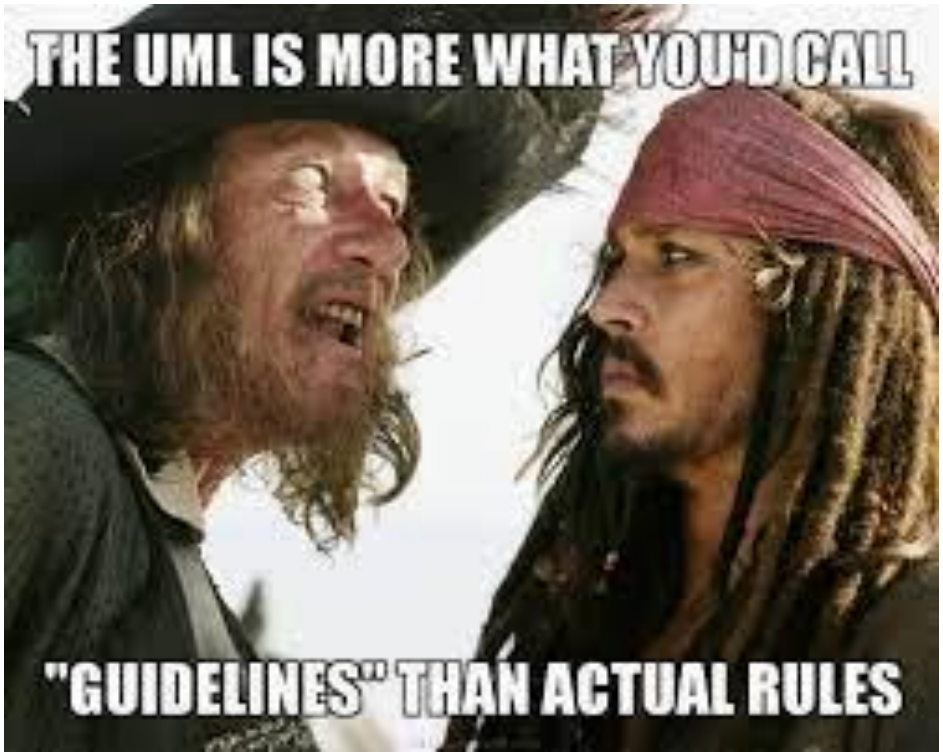
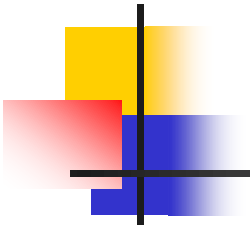
Domain Model





Domain Model

- Why:
  Domain modeling helps us to identify the relevant concepts and ideas of a domain
- When:
  Domain modeling is done during object-oriented analysis

- The domain model is created during object-oriented analysis to decompose the domain into concepts or objects in the real world
- The model should identify the set of conceptual classes
  (The domain model is iteratively completed.)
- It is the basis for the design of the software

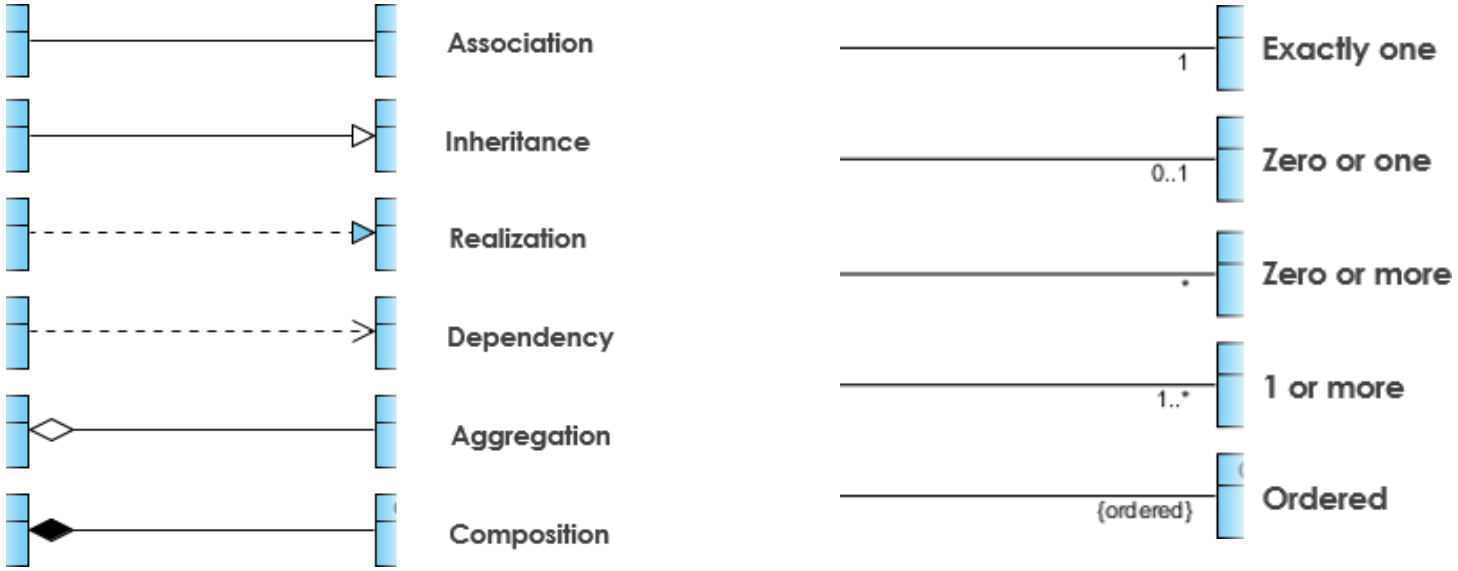
Domain Model

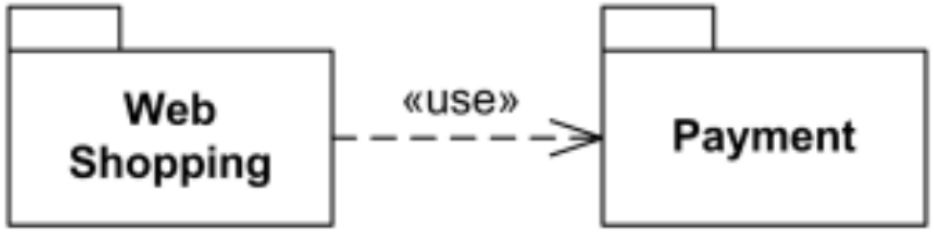
- **Conceptual classes** are ideas, things or objects in the domain
  A conceptual class has a symbol representing the class, an intension and an extension that defines the set of examples to which the conceptual class applies.
  - However, **no operations are defined in domain models**
  - Only …
    - domain objects and conceptual classes
    - associations between them
    - attributes of conceptual classes

To visualize domain models the UML class diagram notation is used.

Visualizing the Domain Model

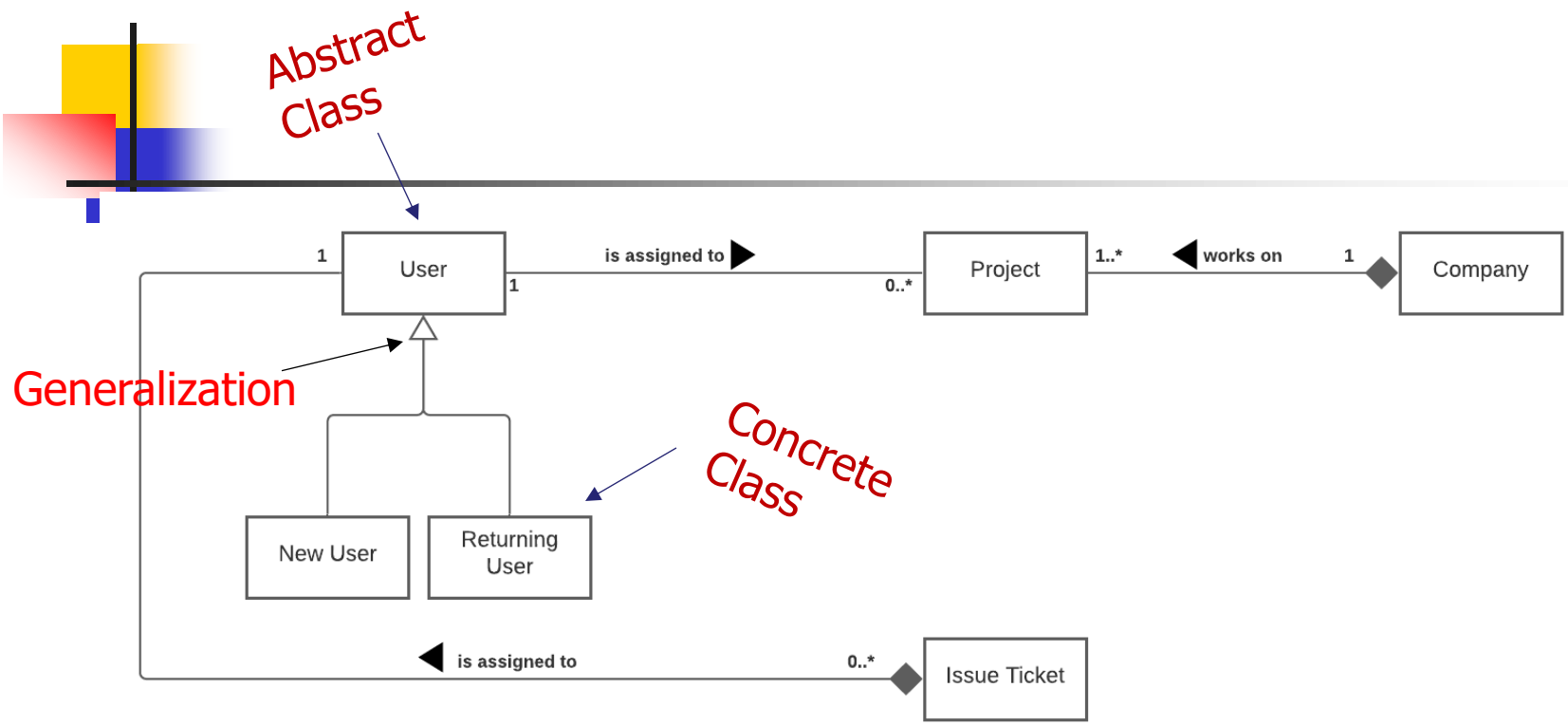
Notation


Association
Inheritance
Realization
Dependency
Aggregation
Composition
1
Exactly one
0..1
Zero or one
*
Zero or more
1..*
1 or more
{ordered}
Ordered



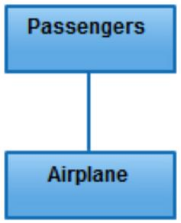
Web Shopping package uses (depends on) Payment package.

A dependency is a connection between dependent and independent model elements. It exists between two elements if changes to the definition of one element (the server or target) may cause changes to the other (the client or source)



Abstract and Concrete Classes

All classes can be designated as either abstract or concrete. Concrete is the default. This means that the class can have (direct) instances. In contrast, abstract means that a class cannot have its own (direct) instances. Abstract classes exist purely to generalize common behaviour that would otherwise be duplicated across (sub)classes.



Association

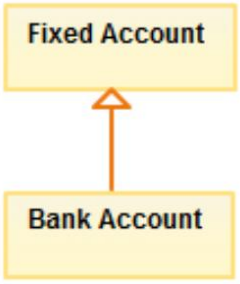
just any logical connection or relationship between classes

Aggregation

the formation of a particular class as a result of one class being aggregated or built as a collection.

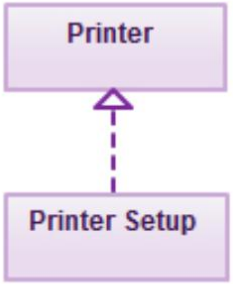
Composition

is very similar to the aggregation relationship. with the only difference being its key purpose of emphasizing the dependence of the contained class to the life cycle of the container class. That is, the contained class will be obliterated when the container class is destroyed.



Inheritance

a type of relationship wherein one associated class is a child of another by virtue of assuming the same functionalities of the parent class. In other words, the child class is a specific type of the parent class.



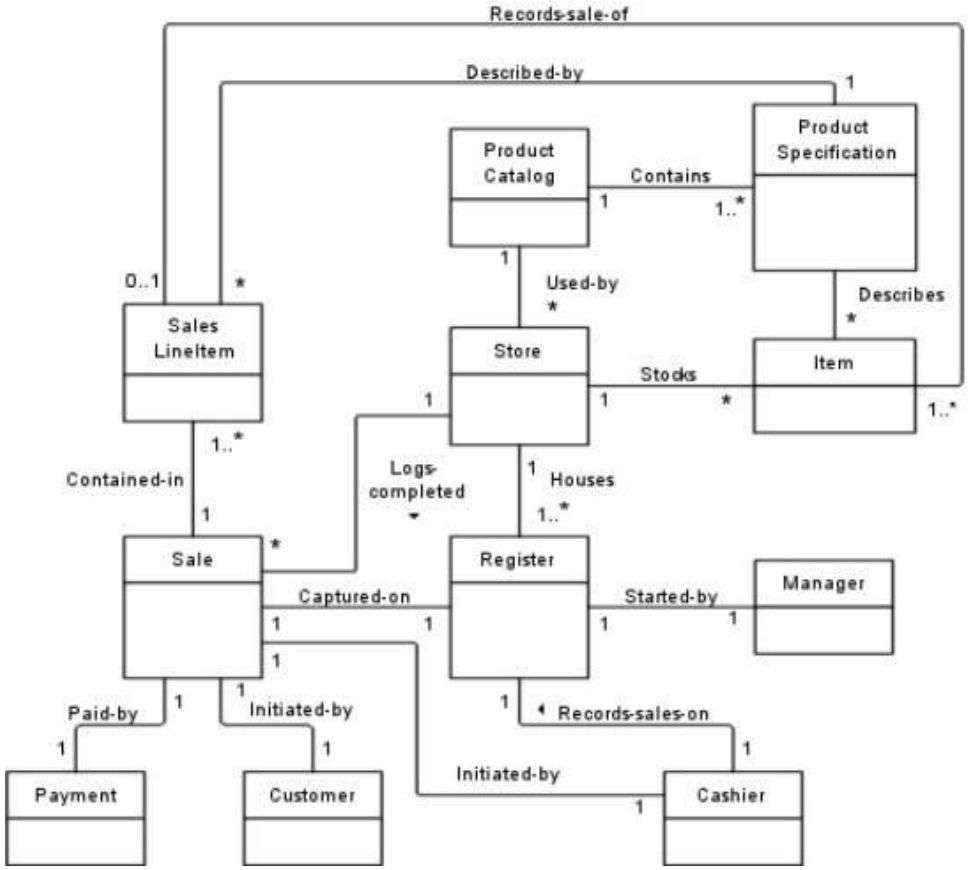
Realization

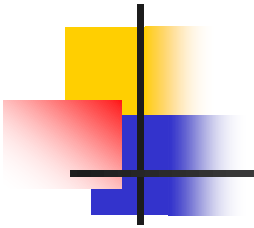
denotes the implementation of the functionality defined in one class by another class. In the example, the printing preferences that are set using the printer setup interface are being implemented by the printer.

So for example imagine a store. For that store you want to build a brand new Point Of Sale system (lets call it POS system).
A POS system is a computerized application used to record sale and handle payments. So you focus on the domain of the POS system.

Now you will conceptualize the objects that will be used for this system. So you will get objects like: Sale, Payment, Register, Item etc.
In a domain model you model these objects and draw associations between them so that you have a high level idea how this system will work.

An example of the POS domain model will be like this:





How to create a Domain Model

- Find the conceptual classes
  - Strategies:
    - Reuse or modify an existing model
    - Use a category list
    - Identify noun phrases
- Draw them as conseptual classes in a UML diagram
- Add associations and attributes

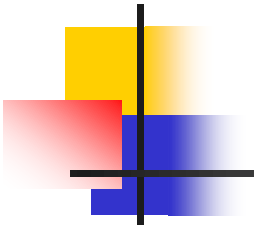
Use the domain vocabulary; e.g. a model for a library should names like "Borrower" insteed of customer

Use a conceptual class category list

Make a list of candidate concepts

Category List

Conceptual Class Category	Classes (for the POS system)
Business transactions...	Sale, Payment
Transaction line items...	SalesLineItem
Product or service related to a transaction or transaction line item.	Item
Where is the transaction recorded?	Register
Roles of people or organizations related to the transaction; actors in use cases.	Cashier, Customer, Store
Place of transactions.	Store
Noteworthy events, often with a time or place that needs to be remembered.	Sale, Payment
...	...



Use noun phrases identification

Identify noun phrases

- Identify the nouns and noun phrases in textual descriptions of a domain and consider them as candidate conceptual classes or attributes
- A mechanical noun-to-class mapping isn´t possible; words in natual languages are ambiguas; e.g. the same noun can mean multible things and multible nouns can mean the same thing

Use cases are an excellent source for identifying conceptual classes

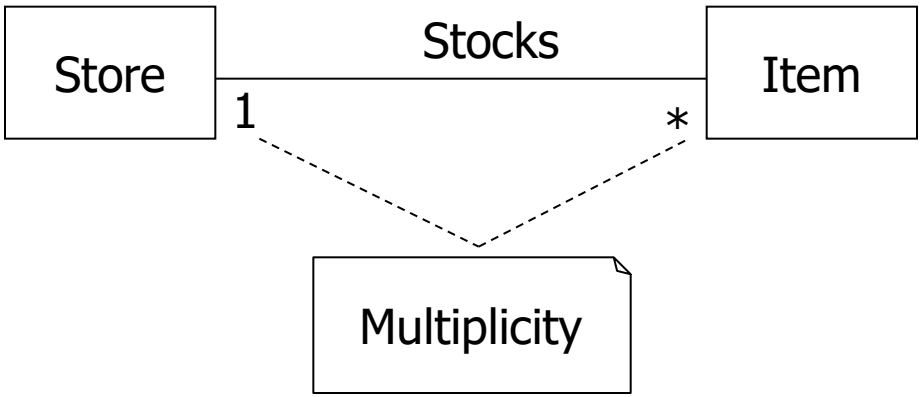
Identify noun phrases to find the conceptual classes.

Process Sale: A customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each item. The system present a running total and line-item details. The customer enters payment information, which the system validates and records. The system updates the inventory. The customer receives a receipt from the system and then leaves the store with the items.

Identified candidate conceptual classes: **Customer**, **Item**, **Cashier**, **Store**, **Payment**, **Sales Line Item**, **Inventory**, **Receipt**, **Sale**.

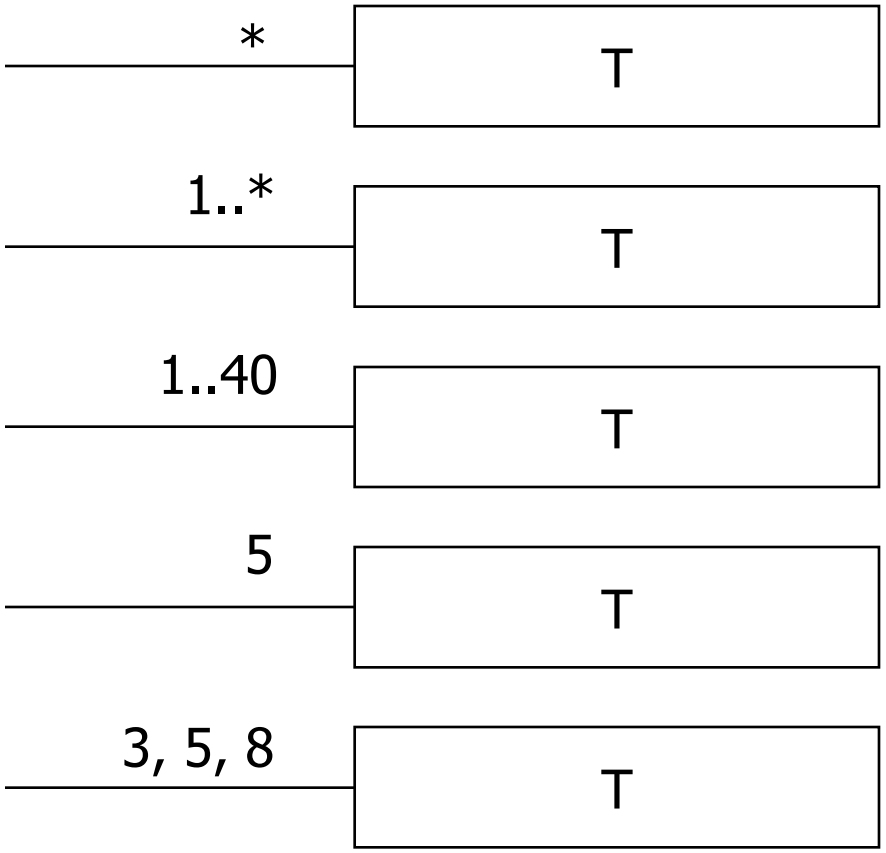
A receipt is just a report of a sale and a payment...

Multiplicity



- Multiplicity defines how many instances of a type A can be associated with one instance of a type B, at a particular moment in time.
- For example, a single instance of a Store can be associated with “many” (zero or more) Item instances.

Multiplicity



Zero or more; "many"

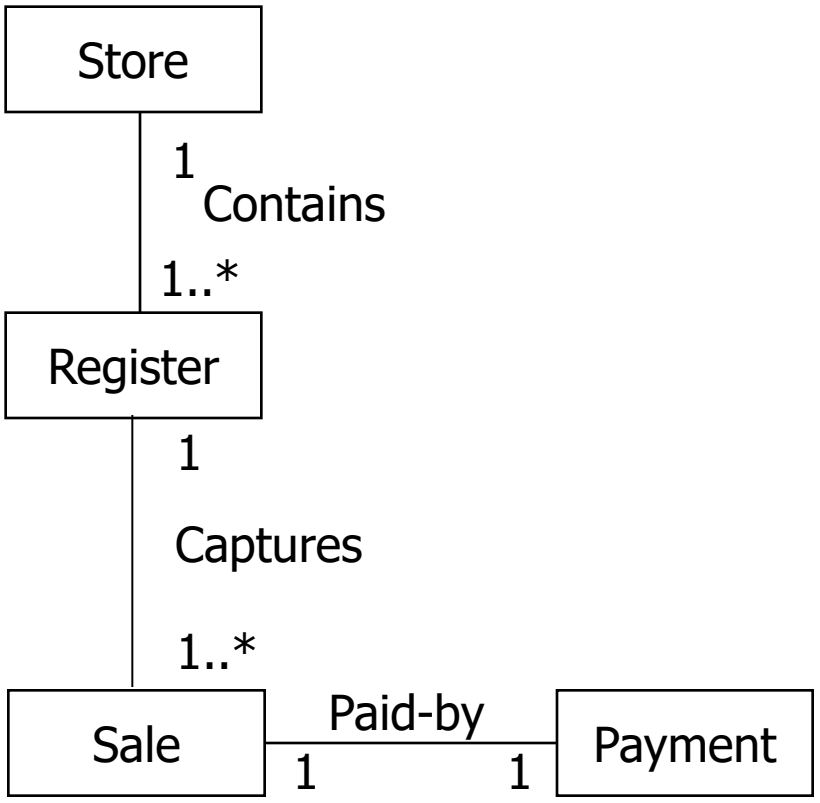
One or more

One to forty

Exactly five

Exactly three, five or eight.

Naming associations



- Name an association based on a `TypeName-VerbPhrase-TypeName` format.
- Association names should start with a capital letter
- A verb phrase should be constructed with hyphens
- The default direction to read an association name is left to right, or top to bottom.

Domain Model